Serverless

Page No:
Date.

→ Serverless doesn't means no running servers

It means you don't manage them

→ Just deploy code/function

→ Initially serverless == Faas [Function as a service]

→ Combined with Autoscaling feature

Serverless service in AWS:-

→ Lamda
→ DynamoDB
→ AWS Cognito
→ AWS API Gateway
→ Fargate
→ S3
→ SNS
→ SQS
→ Kinesis Data base
→ Step Function
→ Aurora serverless

- Athena
- Amazon Quicksight

very very Powerful

EC2 [ ] vs Lamda [λ]

- Virtual servers on cloud
- Limited by RAM & CPU
- Continously running
- Scaling mean intervention to add/remove servers.

- Virtual function
- no server to manage
- Limited by time-shot executions
  - Max 15 minutes
- Run on demand
- Scaling is automated

- Pricing is way less.
- Integrated with so many AWS service
- Integrated with many programming language
- Easy monitor through cloudwatch

- Can increase RAM upto 10 GB which automatically improve CPU & network simultaneously

- Used for cloud cost optimization & Cron Jobs.
- Data processing automated like resize image

- Pay-as-you Go in Business
ie
- Run instance only when order is placed & added to respective database
- saves costs.

AWS Lambda -> Limits

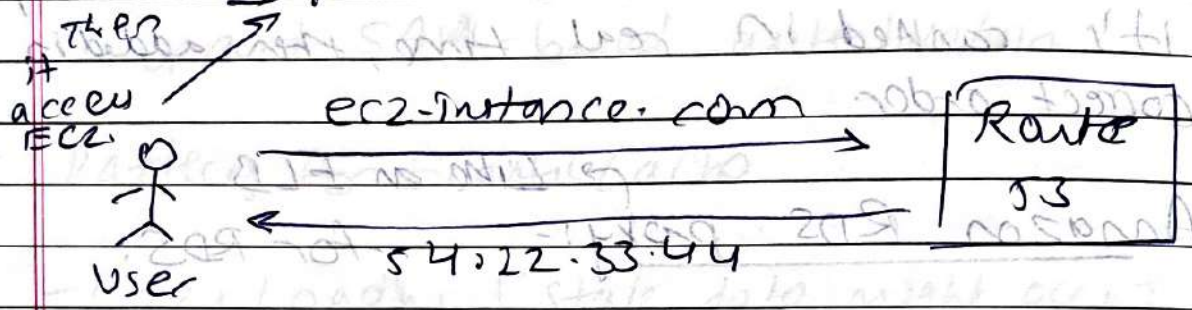
ie when it can't be used?

↳ Exam ask Questions like we need 30 GB ram

AWS Route 53

- A highly available, scalable fully managed & ~~out~~ authoritative DNS
  [the customer can update the DNS records.]

- Eg. You have an EC2 Instance with IP - 54.22.33.44



- Route 53 is also a Domain Registrar

- Why Route 53 name?
  53 is refrence to the traditional DNS port.

- Each record contains:

Domain/subdomain name - eg. - example.com
Record Type - eg. A, AAAA, CNAME
Value - eg. 1.2.3.4

Routing policy - how Route 53 respond to queries

TTL - amount of time record is cache at DNS resolver
↑
Time to Live.

Record Type:

→ A - maps host name to IPV4 address
→ AAAA - maps host name to IPV6 address

→ CNAME - maps hostname to another hostname
- The target is a domain name which must have A or AAAA record.

- can't create a CNAME record for the top node of a DNS namespace.

→ NS - Name servers for Hosted Zone.

Hosted zone:

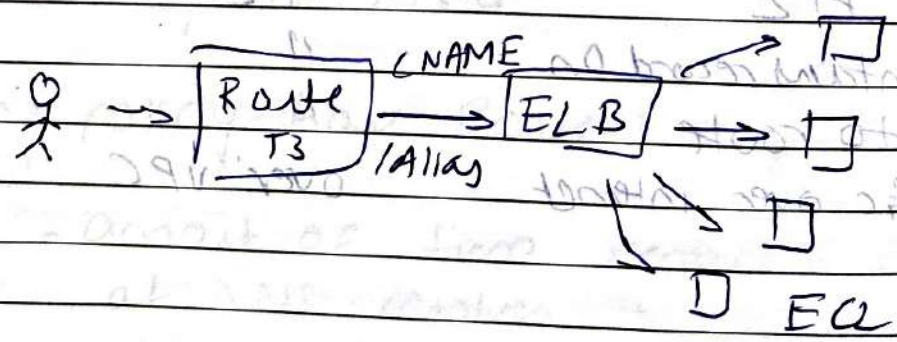
- A container for records that define how to route traffic to a domain } contains data

Hosted zone

Public HZ — contains record on How to route traffic over internet

Private HZ — over VPC

CNAME	vs	ALIAS
One hostname to other hostname		One hostname to a AWS resource
Only for Non root domain		For both Root Level & Non-root level
Eg: example.com		Eg:
You can create CNAME www.example.com only or app.example.com not something.com		can create something.com
		- You can't set TTL
		- Used for ELB, CloudFront Distributions, API Gateway, Elastic Beanstalk, S3 websites
		- You can't set alias to EC2 instance



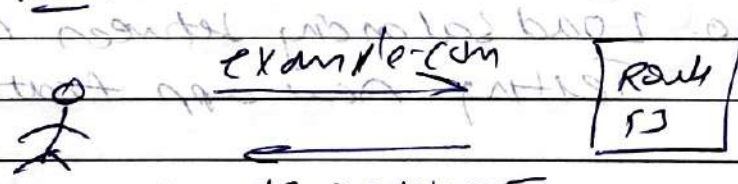
Routing Policies:

Define how Route53 responds to DNS queries
i.e repeat-1 DNS query.

Routing policies are - 1.] Simple
2.] Weighted
3.] Failover
4.] Latency based
5.] Geolocation
6.] Multi-value Answer
7.] Geoproximity

Routing policy

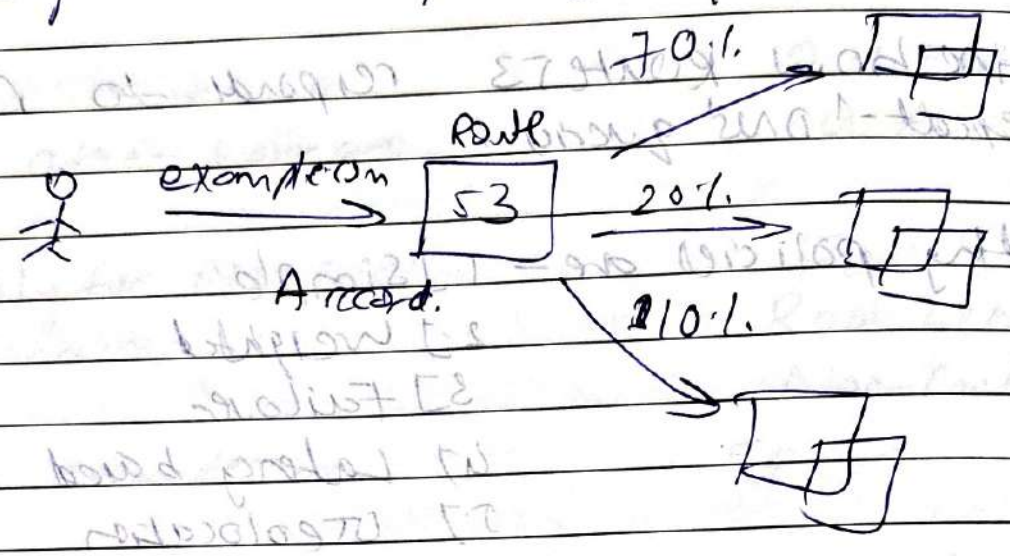
1.] Simple Routing:



A ~~name~~ record.

- If we have set multiple A records let's say '3' it returns '3' IPs & one is selected at randomly simple Routing policy.
- can't associate health checks

2.] Weighted routing policy (weighted)



- DNS records must have the same record type.
- Can be associated with health checks.

Eg. Use case. Load balancing between region, Testing new app features.

- Not necessary that sum is 100%.

3.] Latency routing policy

- Redirect to the resource that has the least latency close to us.

- Super helpful when latency for users is a priority.

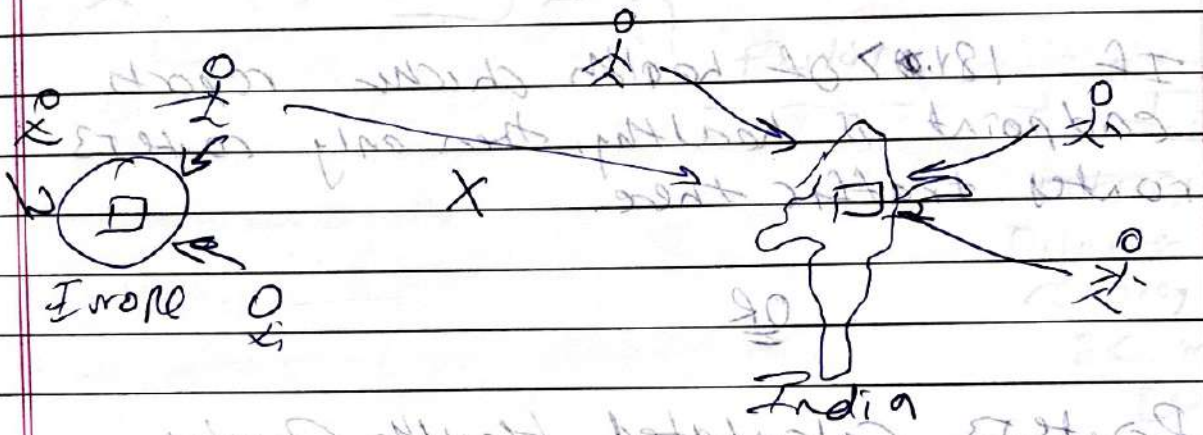
Latency: Time delay it takes for user's request to travel from ~~to~~ their location to AWS data center & back.

Low latency & Fast response

Use case: When resource are in multiple region

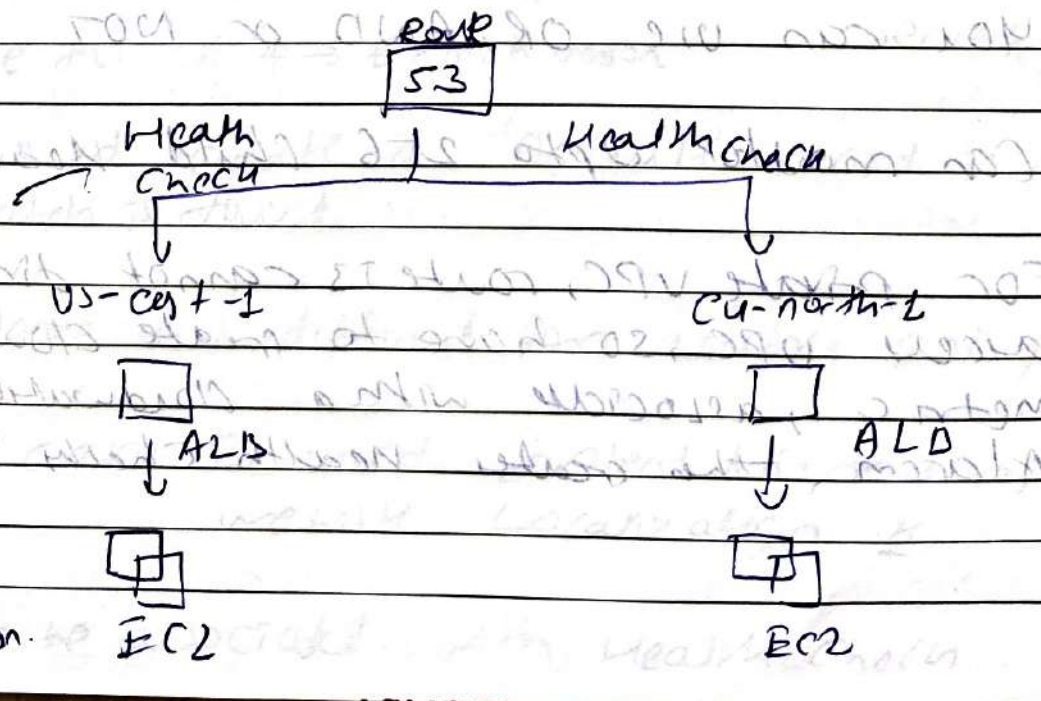
- User gets data from the nearest region, reducing latency.

- can be associated with health checks



Route 53 - Health checks

HTTP health checks are only for public resources.



If not healthy, don't send traffic in that region.

→ Health checks have their own metrics.

→ WOW! → About IT global health checker will ~~ca~~ check endpoint whether it is healthy or not! [Respond code - 200 Then healthy]

- Health/Unhealthy threshold - 3 (default)
- Interval - 30 second.

If 18% of health checker reports endpoint is healthy, then only route 53 routes traffic there.

OR

Route 53 Calculated Health Checks

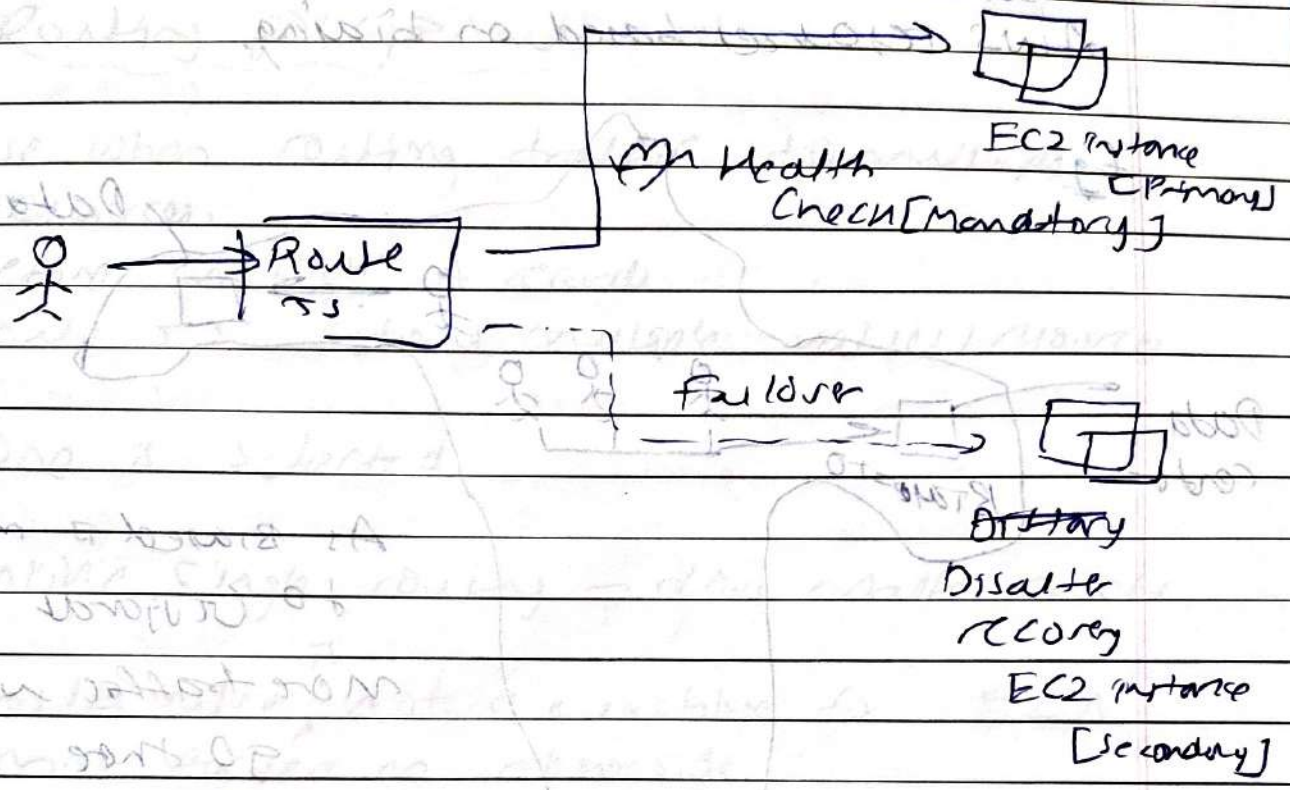
Parent → Combined the results of multiple Health Checks into a single Health Check.

- You can use OR, AND or NOT

- Can monitor upto 256 child Health checks

→ For private VPC, route 53 cannot directly access VPC, so have to create cloudwatch metric, associate with a cloudwatch Alarm, then create Health Check.

Routing Policy – Failover



Routing Policy – Geolocation:

Hindi version

If user from India → Route here to India servers

If user from Europe → Route to Europe servers

Else to US A → English version

User with version

- Should create a 'Default' record if no match is found

- Hence it is different from Latency policy.

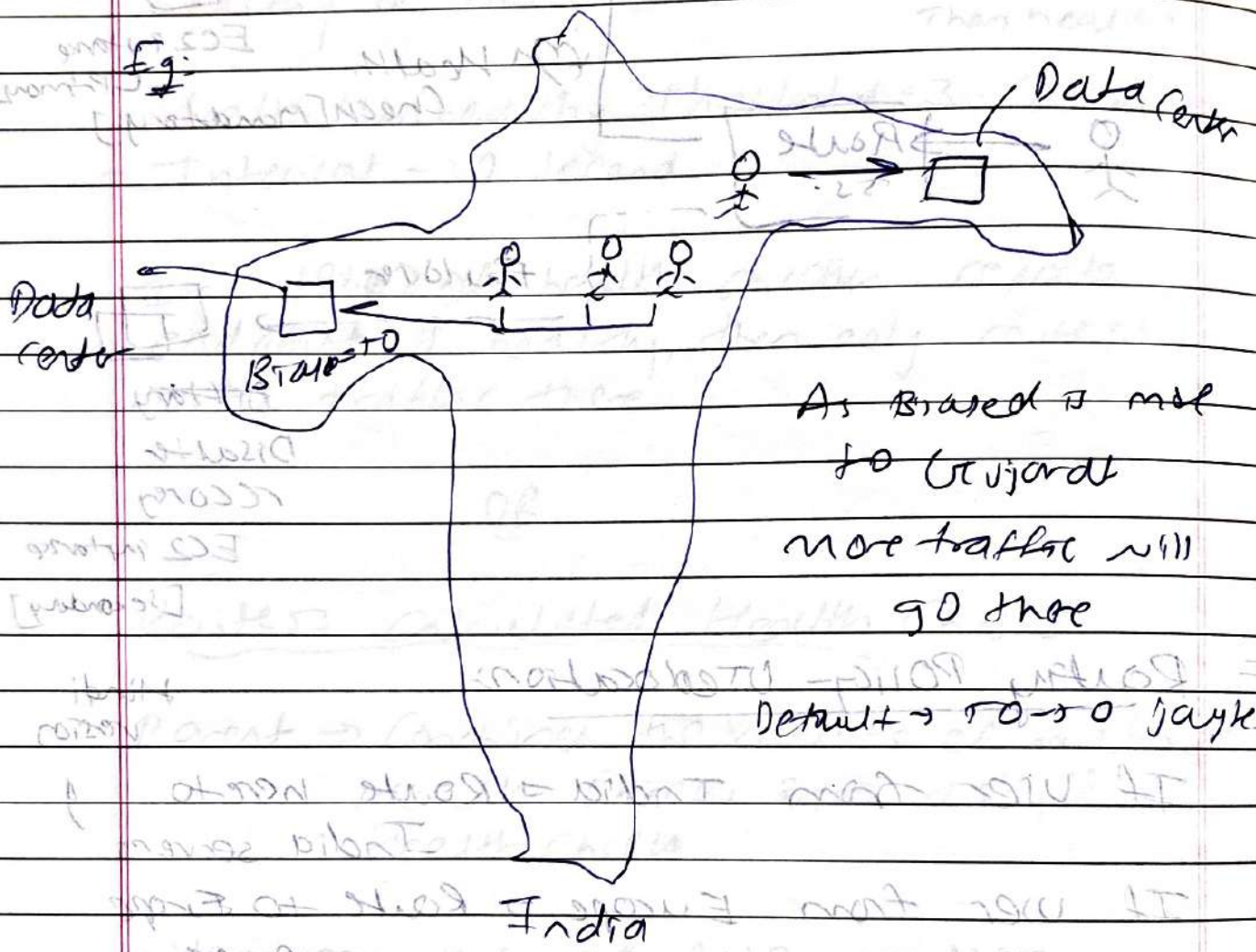
Use case:- Restrict content distribution, website Localization etc

- Can be associated with Health Checks.

Routing policy → Geoproximity

- Routes on basis of user & resource.
- Used to shift traffic to a particular AWS resource based on biasing.

Eg:



As Biased is more to Gujarat more traffic will go there

Default → +0 → 0 Jayk.

Routing policy → IP-based Routing.

- Routing based on IP address
- CIDR block is given to Route 53
- routes IP address in the CIDR block to it given destination
- CIDR → Group of IP address range

Eg: 203.0.113.0/24 0 to 255 routes!

Page No:
Date.

Use case: Optimize performance, reduce network cost.

Routing policy → Multi value

- Use when routing traffic to multiple resources.
- Define multiple A records.
- Route 53 returns multiple values/resources
- One is selected
- Unlike single routing → does health checks
- Multi value, not a substitute for ELB
- more focused on client side.

Architect model

1.] Route53

- "Which region or resource should I send user to?"

- Based on latency
- Primary fails, secondary takes over
- Health check fails, do not send traffic on that region
- Global → cannot do autoscaling

+

2.] ELB → cannot route on basis of latency

- Region scoped (multi AZ possible.) → "Which EC2 container should I send traffic to, within the region?"

- Distributes over 1000s of Instance
- ~~Replace~~ Replace dead instance with health one
- Combines with Autoscaling group

- Routes /api/* to one microservice and web/* to another.